

<u>CL-2006</u> <u>Operating System Lab</u>	<u>LAB -02</u> <u>Creating, Compiling and</u> <u>Executing C/C++ programs using</u> <u>gcc/g++ Compilers and Makefile</u>
---	--

National University of Computer and Emerging Science
Spring 2026

Table of Content

Objective.....	3
Background.....	3
Basic Compilation Command:.....	3
Compilation Process:.....	3
Compilation Options:.....	4
Multi-file Compilation:.....	4
Command Line Arguments:.....	4
Makefile:.....	5
Naming:.....	5
Automatic Variables:.....	5
Simple Example of Makefile:.....	6
Makefile 1.....	6
Makefile 2.....	7
Makefile 3.....	7
Makefile 4.....	8
Recompiling the Linux Kernel:.....	9
Installing Dependencies:.....	9
Download Kernel Sources:.....	9
Configure Kernel:.....	10
Compile Kernel:.....	10
Install New Kernel:.....	11
Update GRUB Configuration:.....	11
Reboot:.....	11

Objective

After completing this lab, you will be able to:

- Compile and run C and C++ programs using gcc and g++
- Understand the compiler process (source → executable)
- Use command line arguments in programs
- Create and use Makefiles for multi-file projects
- Get a conceptual overview of Linux kernel compilation

Background

Basic Compilation Command:

For C++ Programs

```
g++ source.cpp -o output
```

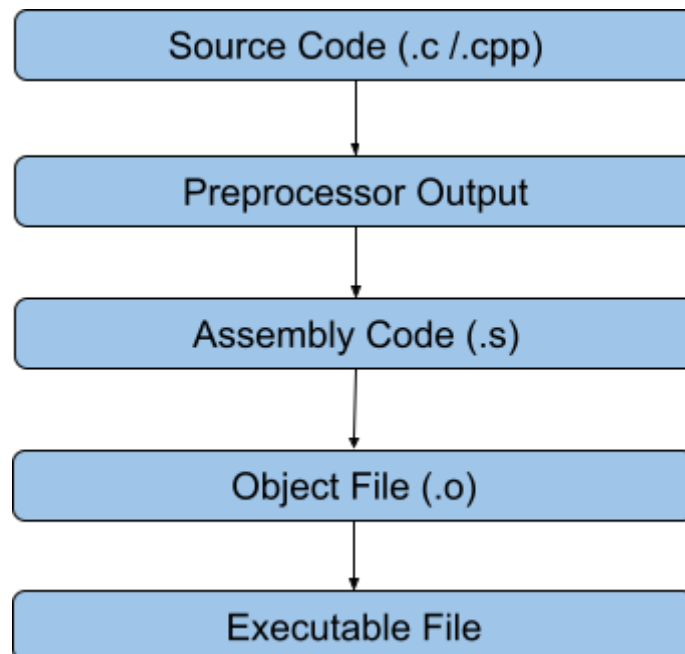
For C Programs:

```
gcc source.c -o output
```

The output file has no extension in Linux.

Compilation Process:

Every C/C++ program goes through multiple stages:



Each source file gets its **own** .o object file.

Compilation Options:

Option	Purpose
- E	Run only preprocessor
- S	Stop after assembly generation
- c	Stop after object file creation
none	Complete process → executable

Multi-file Compilation:

g++ main.cpp -c -o main.o
g++ math.cpp -c -o math.o
g++ main.o math.o -o my_program

my_program is your final executable.

Command Line Arguments:

Command line arguments are a way to pass data to the program. Command line arguments are passed to the main function. Suppose we want to pass two integer numbers to the main function of an executable program called a.out. On the terminal write the following line:

```
./a.out 10 20
```

./a.out is the usual method of running an executable via the terminal. Here 1 and 22 are the numbers that we have passed as command line arguments to the program. These arguments are passed to the main function.

In order for the main function to be able to accept the arguments, we have to change the signature of main function as follows:

```
int main(int argc, char *arg[])
```

Here:

- **argc** is the counter. It tells how many arguments have been passed.
- **arg** is the character pointer to our arguments.

NOTE: argc is always ≥ 1 because the program name is included.

argc in this case will not be equal to 2, but it will be equal to 3. This is because of the name ./a.out is also passed as command line argument. At index 0 of arg, we have ./a.out; at index 1,

we have 1; and at index 2, we have 22. Here 1 and 22 are in the form of character string, we have to convert them to integers by using a function **atoi**. Suppose we want to add the passed numbers and print the sum on the screen:

```
#include <iostream>
#include <cstdlib>
using namespace std;

int main(int argc, char *arg[]) {
    int a = atoi(arg[1]);
    int b = atoi(arg[2]);
    cout << a + b;
}
```

Makefile:

- Makefile is a text file that tells the make utility how to compile and build a program automatically. It defines which files depend on each other and which commands should run when a file is changed, so only the necessary parts of the program are recompiled.
- A Makefile is used to automate the compilation process by specifying dependencies and building rules for a program.
- A Makefile helps us compile large programs easily without typing long commands again and again.

Naming:

makefile or Makefile are standard but other names can be also used:

\$make

or

\$make -f filename

Where filename is the name of your file is not “makefile” or “Makefile”

Automatic Variables:

Automatic variables are used to refer to specific part of rule components. eval.o : eval.c eval.h

\$@	The name of the target of the rule (eval.o).
\$<	The name of the first dependency (eval.c).
\$^	The names of all the dependencies (eval.c eval.h).
\$?	The names of all dependencies that are newer than the target make

Simple Example of Makefile:

Let's start off with the following three files, `hellomake.c`, `hellofunc.c`, and `hellomake.h`, which would represent a typical main program, some functional code in a separate file, and an include file, respectively.

hellomake.c	hellofunc.c	hellomake.h
<pre>#include <hellomake.h> int main() { // call a function in another file myPrintHelloMake(); return(0); }</pre>	<pre>#include <stdio.h> #include <hellomake.h> void myPrintHelloMake(void) { printf("Hello makefiles!\n"); return; }</pre>	<pre>/* example include file */ void myPrintHelloMake(void);</pre>

Normally, you would compile this collection of code by executing the following command:

```
gcc -o hellomake hellomake.c hellofunc.c -I.
```

This compiles the two `.c` files and names the executable `hellomake`. The `-I.` is included so that `gcc` will look in the current directory (`.`) for the include file `hellomake.h`. Without a makefile, the typical approach to the test/modify/debug cycle is to use the up arrow in a terminal to go back to your last compile command so you don't have to type it each time, especially once you've added a few more `.c` files to the mix.

Unfortunately, this approach to compilation has two downfalls. First, if you lose the compile command or switch computers you have to retype it from scratch, which is inefficient at best. Second, if you are only making changes to one `.c` file, recompiling all of them every time is also time-consuming and inefficient. So, it's time to see what we can do with a makefile.

The simplest makefile you could create would look something like:

Makefile 1

```
hellomake: hellomake.c hellofunc.c hellomake.h
    gcc -o hellomake hellomake.c hellofunc.c -I.
```

If you put this rule into a file called `Makefile` or `makefile` and then type `make` on the command line it will execute the compile command as you have written it in the makefile. Note that `make` with no arguments executes the first rule in the file. Furthermore, by putting the list of files on which the command depends on the first line after the `:`, `make` knows that the rule `hellomake` needs to be executed if any of those files change. Immediately, you have solved problem #1 and

can avoid using the up arrow repeatedly, looking for your last compile command. However, the system is still not being efficient in terms of compiling only the latest changes.

One very important thing to note is that there is a tab before the gcc command in the makefile. There must be a tab at the beginning of any command, and make will not be happy if it's not there.

In order to be a bit more efficient, let's try the following:

Makefile 2

```
CC=gcc
CFLAGS=-I.

hellomake: hellomake.o hellofunc.o
    $(CC) -o hellomake hellomake.o hellofunc.o
```

So now we've defined some constants CC and CFLAGS. It turns out these are special constants that communicate to make how we want to compile the files hellomake.c and hellofunc.c. In particular, the macro CC is the C compiler to use, and CFLAGS is the list of flags to pass to the compilation command. By putting the object files--hellomake.o and hellofunc.o--in the dependency list and in the rule, make knows it must first compile the .c versions individually, and then build the executable hellomake.

Using this form of makefile is sufficient for most small scale projects. However, there is one thing missing: dependency on the include files. If you were to make a change to hellomake.h, for example, make would not recompile the .c files, even though they needed to be. In order to fix this, we need to tell **make** that all .c files depend on certain .h files. We can do this by writing a simple rule and adding it to the makefile.

Makefile 3

```
CC=gcc
CFLAGS=-I.
DEPS = hellomake.h

%.o: %.c $(DEPS)
    $(CC) -c -o $@ $< $(CFLAGS)

hellomake: hellomake.o hellofunc.o
    $(CC) -o hellomake hellomake.o hellofunc.o
```

This addition first creates the macro DEPS, which is the set of .h files on which the .c files depend. Then we define a rule that applies to all files ending in the .o suffix. The rule says that the .o file depends upon the .c version of the file and the .h files included in the DEPS macro. The rule then says that to generate the .o file, make needs to compile the .c file using the compiler defined in the CC macro. The -c flag says to generate the object file, the -o \$@ says to put the output of the compilation in the file named on the left side of the :, the \$< is the first item in the dependencies list, and the CFLAGS macro is defined as above.

As a final simplification, let's use the special macros \$@ and \$^, which are the left and right sides of the :, respectively, to make the overall compilation rule more general. In the example below, all of the include files should be listed as part of the macro DEPS, and all of the object files should be listed as part of the macro OBJ.

Makefile 4

```
CC=gcc
CFLAGS=-I.
DEPS = hellomake.h
OBJ = hellomake.o hellofunc.o

%.o: %.c $(DEPS)
    $(CC) -c -o $@ $< $(CFLAGS)

hellomake: $(OBJ)
    $(CC) -o $@ $^ $(CFLAGS)
```

Let's break down each line:

1. **CC=gcc** : This line defines a variable `CC` and sets its value to `gcc`, indicating that the GNU C Compiler (`gcc`) will be used for compiling the source files.
2. **CFLAGS=-I.** : This line defines a variable `CFLAGS` and sets its value to `-I.`. `-I` is a compiler option used to specify additional directories to search for header files. However, it seems incomplete here as no directory is specified after `-I`.
3. **DEPS = hellomake.h**: This line defines a variable `DEPS` and sets its value to `hellomake.h`. `DEPS` typically stands for dependencies, and in this case, it specifies the header file(s) that the source files depend on.
4. **OBJ = hellomake.o hellofunc.o**: This line defines a variable `OBJ` and sets its value to `hellomake.o hellofunc.o`. OBJ typically stands for object files, and it lists the object files that need to be generated by compiling the corresponding source files.
5. **%.o: %.c \$(DEPS)**: This is a pattern rule that specifies how to generate an object file (`%.o`) from a corresponding source file (`%.c`) along with its dependencies (`\$(DEPS)`). When invoked, this rule tells `make` that for any `.o` file it needs to create, it can find the corresponding `.c` file and its dependencies in order to compile them.

6. `$(CC) -c -o $@ $< $(CFLAGS)`: This line is the recipe associated with the pattern rule defined above. It specifies the commands to compile a `.c` file into an object file.
 - `$(CC)`: Evaluates to `gcc`, indicating the compiler to be used.
 - `-c`: Instructs the compiler to generate an object file.
 - `-o $@`: Specifies the output file name, where `$@` is a built-in variable representing the target of the rule, which in this case is the object file being generated.
 - `$<`: Represents the first prerequisite of the rule, which is the source file (`%c`).
 - `$(CFLAGS)`: Contains additional compiler flags, though it's incomplete in this Makefile.
7. `hellomake: $(OBJ)`: This line defines a target `hellomake` that depends on the object files listed in `$(OBJ)`. It specifies that the executable `hellomake` should be built using these object files.
8. `$(CC) -o $@ $^ $(CFLAGS)`: This line is the recipe associated with the target `hellomake`. It specifies the commands to link the object files into an executable.
 - `$(CC)`: Evaluates to `gcc`, indicating the linker to be used.
 - `-o $@`: Specifies the output file name, where `$@` is a built-in variable representing the target of the rule, which in this case is the executable being generated (`hellomake`).
 - `$^`: Represents all the prerequisites of the rule (`$(OBJ)`), i.e., the object files.
 - `$(CFLAGS)`: Contains additional linker flags, though it's incomplete in this Makefile.

Recompiling the Linux Kernel:

The general steps to recompile the Ubuntu kernel from sources with parameter changes:

```
$uname -r
```

Installing Dependencies:

Install necessary build tools and dependencies. If you have not built a kernel on your system before, there are some packages needed before you can successfully build. You can get these installed with:

```
$sudo apt install build-essential libncurses-dev bison flex libssl-dev libelf-dev fakeroot
```

```
$sudo apt install dwarves
```

Download Kernel Sources:

Next we need to need to download the kernel source from the official <https://www.kernel.org/> website. Choose the latest long term release and copy the link of the tarball hyperlink. Then use this link to download and unpack the kernel source to your Ubuntu machine:

```
$wget https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.1.73.tar.xz
```

```
$tar -xf linux-6.1.73.tar.gz
```

```
$ cd linux-6.1.73
```

Configure Kernel:

We start by copying existing configuration as new configuration.

```
$cp -v /boot/config-$(uname -r).config
```

Adjust the configuration using a menu-based interface:

```
$make menuconfig
```

Use the arrows to make a selection or choose Help to learn more about the options. When you finish making the changes, select Save, and then exit the menu.

Disable the conflicting security certificates by executing the following commands below:

```
$ scripts/config --disable SYSTEM_TRUSTED_KEYS
```

```
$ scripts/config --disable SYSTEM_REVOCATION_KEYS
```

```
$ scripts/config --set-str CONFIG_SYSTEM_TRUSTED_KEYS ""
```

```
$ scripts/config --set-str CONFIG_SYSTEM_REVOCATION_KEYS ""
```

Compile Kernel:

Build the kernel and modules:

```
$make -j$(nproc)
```

The `-j$(nproc)` option enables parallel compilation, utilizing the number of available processor cores. Press Enter repeatedly to confirm the default options for the generation of new certificates.

Install Modules:

Install the kernel modules:

```
$sudo make modules_install
```

Install New Kernel:

Install the new kernel:

```
$sudo make install
```

This will update the bootloader configuration (GRUB) and copy the new kernel image to `/boot`.

Update GRUB Configuration:

Ensure that GRUB is aware of the new kernel. Run:

```
$sudo update-grub
```

Reboot:

Reboot the system to load the new kernel:

```
$sudo reboot
```

Verify:

After rebooting, check the kernel version:

```
$uname -r
```

Lab Activity

1. Write a C program that accepts student scores (out of 100) as command line parameters and calculates their average grade. Implement robust error checking for missing or incorrectly formatted parameters. For example, if the user runs the program with `./grade_calculator 85 90 78`, it should compute the average of these scores and display it.
2. Write a simple C program that accepts a series of integers as command line parameters, stores them in an array, sorts the array in ascending order, and prints the sorted array to the screen. For example, if the input is `./sort_integers 4 1 3 2`, the output should be `1 2 3 4`.
3. Event Reservation System (Basic) Create a header file named `event_reservation.h` that declares functions for an Event Reservation System. Declare functions for adding and displaying events. Implement these functions in a source file named `event.c`. Provide a simple main function in another source file named `event_main.c` to showcase adding and displaying events. Now compile all classes using `makefile`.
4. Write a C program that accepts integers as command line parameters, stores them in an array, and prints the sum and average of those integers. Implement error checking for missing parameters and non-integer inputs. For example, running `./sum_average 10 20 30` should output the sum (60) and average (20).
5. Write a C program that accepts a series of integers as command line parameters, stores them in an array, computes the missing element in a sequence (assuming the sequence is a continuous range of numbers), and prints the missing element. For example, for input `1 2 4 5`, the program should output 3.